R1D4 Robot User Interface and Swarm Robots

ECSE 458D1/D2: Capstone Design Project



Capstone Group: DP 36

Abonty, Noorjahan - ID: 260953393 (noorjahan.abonty@mail.mcgill.ca)
Bizzarri, Paolo - ID: 261031842 (paolo.bizzarri@mail.mcgill.ca)
Hershkop, Joey - ID: 260983531 (joseph.hershkop@mail.mcgill.ca)
Singhal, Anjali - ID: 261011655 (anjali.singhal@mail.mcgill.ca)

Project Advisor:

Joseph Vybihal (joseph.vybihal@mcgill.ca)

April 11th, 2025

Abstract

Our project is divided into two main components. The first, developed mostly during the first semester, focused on building an Android Application to serve as the facial interface for the R1D4 multifunction robot. Designed for use in a hospital environment, the R1D4 robot autonomously delivers medical supplies between rooms. The Android app, developed using React Native and Expo, enables users to schedule deliveries, confirm pickups, and edit or delete tasks through a tablet interface. A key challenge was enabling communication between the app, an ESP32 microcontroller, the robot, and a Telnet server that governs robot movement. By early in the second semester, we had completed the app and successfully deployed it on the designated tablet device.

The second phase of the project centered on developing a Navigation System for the HoverBot, one of the Swarm Robots. This required a deep understanding of the existing architecture, including the REST API and TCP server. Using Python's OpenCV library, we implemented object tracking and applied parallax correction techniques. A navigation algorithm was then developed, enabling the user to click on a point within a live camera feed to select a destination. The system computes a path and communicates with the robot through the REST API, allowing the HoverBot to reach the selected location. This phase involved extensive testing and integration to ensure reliable navigation performance.

Acknowledgments

We would like to thank Capstone group 17 – Bilar Mokhtari, Walid Aissa, Jules Delabrousse, Zakaria Benhssein. They helped familiarize us with the current state of the R1D4 hardware.
We would also like to thank Pinak Ghosh, Yu Qing Liu, Haoxi Li and Zachary Godden for their documentation, which helped us understand the current state of the HoverBot and greatly supported our initial setup. Pinak Ghosh also provided valuable assistance during the early stages of configuring the HoverBot for the Swarm Robots portion of the second part of our Capstone project.

Table of content

1. List of Abbreviations and/or Notation Used in the Report 4
2. Introduction/Motivation/Objective 5
3. Background 5
3.1 Background for the HoverBot Navigation 5
3.1.1 Hardware Setup 6
3.1.2 TCP Server and REST API Controller 6
3.1.3 OpenCV 7
3.1.4 Required Computer Vision Capabilities and Concepts 7
3.2 Background for the R1D4 App 8
3.2.1 Async Storage 8
4. Requirements and/or Problems 8
4.1 Requirements 8
4.2 Constraints 8
5. Design and Results 9
5.1 Design and Implementation of the HoverBot Navigation System 9
5.1.1 Overarching Design Philosophy and Goals 9
5.1.2 System Architecture and Data Flow 10
5.1.3 Detailed Implementation Choices and Justifications 10
5.2 Results of HoverBot Navigation 12
5.3 Design and Implementation of the R1D4 Interface 13
5.3.1 Delivery Status 13
5.3.2 USB Serial Package 14
5.3.3 Design Decisions for the R1D4 App 14
5.4 Results of R1D4 App 15
5.5 Setup Instructions for HoverBot Navigation System 15
5.5.1 Prerequisites 15
5.5.2 Hardware Initialization 16
5.5.3 TCP Server 16
5.5.4 Python Script 16
5.6 Setup Instructions for R1D4 App 18
6. Impact on Society and the Environment 19
6.1 Use of Non-Renewable Resources 19
6.1.1 HoverBot Navigation System 19
6.1.2 R1D4 Robot User Interface 19
6.2 Environmental Benefits 19
6.2.1 HoverBot Navigation System 19
6.2.2 R1D4 Robot User Interface 19
6.3 Safety and Risk 20
6.3.1 HoverBot Navigation System 20

6.3.2 R1D4 Robot User Interface 20
6.4 Benefits to Society 20
6.4.1 HoverBot Navigation System 20
6.4.2 R1D4 Robot User Interface 20
7. Report on Teamwork **20**
7.1 Team Collaboration 20
7.2 Individual Contributions 21
8. Conclusion **22**
8.1 Next Steps for HoverBot Navigation 22
8.1.1 Hardware 22
8.1.2 Software 23
8.2 Next Steps for R1D4 App 25
9. References **26**
10. Appendices **26**

1. List of Abbreviations and/or Notation Used in the Report

ESP32: Espressif32
TCP: Transmission Control Protocol
IP: Internet Protocol
REST API: Representational State Transfer Application Programming Interface
HTTP: HyperText Transfer Protocol
HSV: Hue, Saturation and Value
ROI: Regions of Interest

2. Introduction/Motivation/Objective

For our Capstone project, we worked on two separate projects in the first and second semester. In the first semester we focused on creating the R1D4 Android App interface. The R1D4 was created to be used in a hospital setting where doctors and nurses could use the robot to deliver prescription medications around the hospital. The robot itself was worked on by another group and while we created the front end system. Our system allowed the user to schedule, edit and confirm deliveries by the robot. This system involved building an android application and a communication network between the robot (ESP32), the android tablet and a telnet server running on a laptop.

Our second project involved adding onto the existing swarm robot project started by previous teams. The swarm robots is a research project with the goal of developing several robots that communicate with a central server. These robots should be able to receive commands individually and then navigate a room. Our addition to this project was creating a computer vision system that would be the first steps in developing navigation for this project. Previously, each robot was unable to navigate without a human giving individual commands from the terminal. After our work, we were able to institute a computer vision based navigation system for one of the robots in the swarm project, namely the HoverBot.

Let's continue by expanding upon the HoverBot navigation system goals and objectives. We can split our goal into two subgoals. Firstly, we needed to create a navigation algorithm that can make real time navigation decisions for the robot and send the required commands. Secondly, we need to develop a computer vision system that can recognize an arbitrary robot and track its position within a frame that the user has set within the camera feed. Additionally, we had to fix lots of bugs and issues with the existing hardware of the HoverBot so that it was working well enough for us to make these new additions.

The "why" behind this new system is simple; we wanted to create an easy way to centralize navigation commands for all the swarm bots. When we inherited the swarm robot project, all the robots were set up to receive command-line prompts to move forward, backward, etc. There was no centralized navigation system where decisions could be made to navigate these robots in a given setting. This was the central problem that we set out to solve. Because of many hardware related issues with the rest of the robots, we decided to set the success criteria for this project as getting the navigation system working for only the HoverBot, but creating it in a modular fashion, so that future groups can add the rest of the robots to the system.

Since we made very few additions to the R1D4 project since our mid project presentation, we are going to focus on the HoverBot navigation system for the majority of this report and only mention the new additions/progress made to the app. However, we will also include a new section titled Setup Guide which will include documentation on how to set up the R1D4 app. Under this section, we will also include documentation to set up the computer vision system and hardware for the HoverBot.

3. Background

3.1 Background for the HoverBot Navigation

3.1.1 Hardware Setup

The hardware of the HoverBot system is primarily built around an ESP32 microcontroller, which operates on top of a custom built ESP32 control unit. This ESP32 unit functions as the central control unit, interfacing with transistor-based inverter circuits that manage brake and directional control signals through 2N2222A NPN transistors. These transistors invert the signals from the ESP32, effectively turning active-low signals into active-high for the braking and directional mechanisms. This is to avoid driving the motor control pins directly from the ESP32. This creates concerns surrounding current supply and can cause issues to the ESP32. Therefore, the system is structured so that when the ESP32 wants to trigger a brake or change direction, it will send a low signal, causing the inverter to output a high signal to activate the control line.

Motor control within the HoverBot system involves dedicated motor controllers for each motor. These controllers receive brake and direction signals from the ESP32 control unit and translate them into appropriate three-phase drive signals to operate the motors. Hall effect sensors provide positional feedback from the motors to the controllers, ensuring accurate control of the motor's rotation and speed. The power system utilizes a 36V battery that feeds into a regulated power distribution network. It features a 12V DC-DC converter, which powers the motor controllers, and a 5V DC-DC converter, supplying the ESP32 control unit. Comprehensive grounding ensures consistent reference points across all hardware components. The design includes protective measures like a 2A fuse in the charging circuit and careful voltage stepping to ensure safe and efficient operation.[1]

3.1.2 TCP Server and REST API Controller

The TCP Server listens on specified ports and IP addresses, establishes TCP sessions with clients, and processes incoming data, supporting multithreaded processing for high-volume data handling [2].
It is a java server enabling user-robot communications using TCP connections. Commands are entered via the command line or using the Python REST client, processed by the server and sent to the recipient robot, which executes it and sends back a response, which is processed again and made viewable to the user. The TCP server is like a traffic controller ensuring smooth, reliable message delivery between users and robots in real time.

The server listens for connections from robots on port 50050. It can receive commands to move forward, backward, turn left, turn right, list robot IDs, ping the robot and change the robot ID.
The REST API provides an HTTP-based interface that users or external clients (Python script) can use to send commands to robots and listens for HTTP requests on 8080. Each command has its own HTTP endpoint (URL) that accepts its corresponding HTTP method (like POST or GET) and optionally (for some) a request body. The details for these commands and the corresponding REST API endpoints can be found at *TcpServer/README.md*.

The client (Python script) sends an HTTP request (POST - /forward with JSON like {"id":1, "param":123.456}) to the REST API. The API uses a DTO (Data Transfer Object), like MessageRequestDto, to validate and structure the incoming data (converting JSON into a Java object). The REST API then creates a Message object and forwards it to the TCP Server, which queues the command for the target robot. The TCP Server translates the Message into low-level instructions (raw bytes or a robot-specific protocol) and sends them over TCP to the robot, which executes the action.

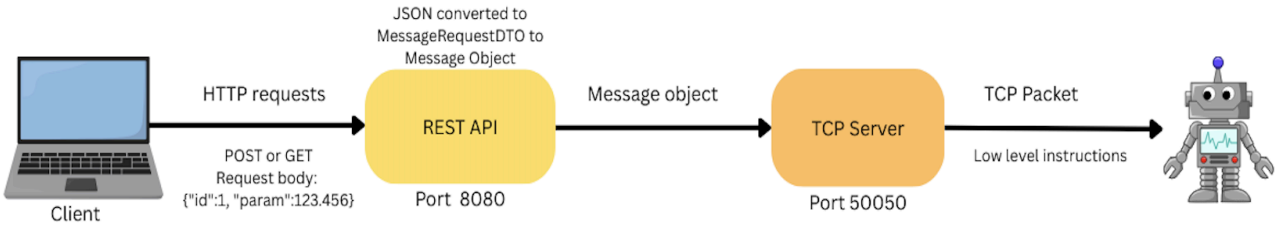


Figure 1: Interaction between the client, REST API, TCP server and the robots.

3.1.3 OpenCV

A fundamental requirement for any autonomous mobile robot, including the HoverBot, is the ability to perceive its environment and determine its own state within it. This process, often termed localization (determining position) and pose estimation (determining position and orientation), is required for navigation tasks such as moving to a target destination. Here, we rely on computer vision to achieve this perception. The primary tool selected for this task is the Open Source Computer Vision Library (OpenCV).

OpenCV is a comprehensive, open-source library focused on real-time computer vision and machine learning applications. It provides thousands of optimized algorithms for a vast array of image and video processing tasks. These range from basic operations like reading, writing, and manipulating images to complex analysis such as object detection, feature matching, motion tracking, and 3D scene reconstruction. Key strengths of OpenCV include its computational efficiency (often leveraging hardware acceleration), cross-platform compatibility (Windows, Linux, macOS, mobile platforms), and availability of interfaces for multiple programming languages, notably C++ and Python. For this project, we utilized the widely-used Python binding, accessed through the cv2 module.[3]

3.1.4 Required Computer Vision Capabilities and Concepts

1. Image Acquisition and Display: The ability to interface with a camera, capture sequential frames (video stream), and display processed results in real-time is fundamental.
2. Geometric Transformations: Cameras often view scenes from an angle, introducing perspective distortion. Correcting this requires understanding and applying geometric transformations, specifically perspective warping (using homography), to obtain an accurate, undistorted "top-down" view of the robot's operating area.
3. Color Space Analysis and Thresholding: Reliably identifying objects based on color requires techniques like color thresholding, which are used to isolate pixels belonging to the target objects (the robot's markers).
4. Contour Detection and Analysis: Once target pixels are isolated, contour detection algorithms are needed to find the outlines of the objects (markers). Analyzing these contours allows for the calculation of properties like area and the centroid (center point) of each marker.
5. Feature Extraction and Pose Estimation: Using the geometric properties (centroids) of the detected markers, mathematical calculations (primarily vector geometry, often aided by libraries like NumPy) are employed to determine the robot's overall position (midpoint) and orientation (angle) within the camera's field of view.
6. Image Annotation: Providing visual feedback is essential for debugging and monitoring. This involves using OpenCV's drawing functions to overlay information such as detected objects, calculated positions, orientation vectors, and target destinations, directly onto the video frames.

7. User Interaction: Enabling user input directly via the display window (e.g., using mouse clicks for calibration or setting targets) requires understanding OpenCV's event handling mechanisms.

3.2 Background for the R1D4 App

Refer to the Mid-Project Report for more explanation of all the components.

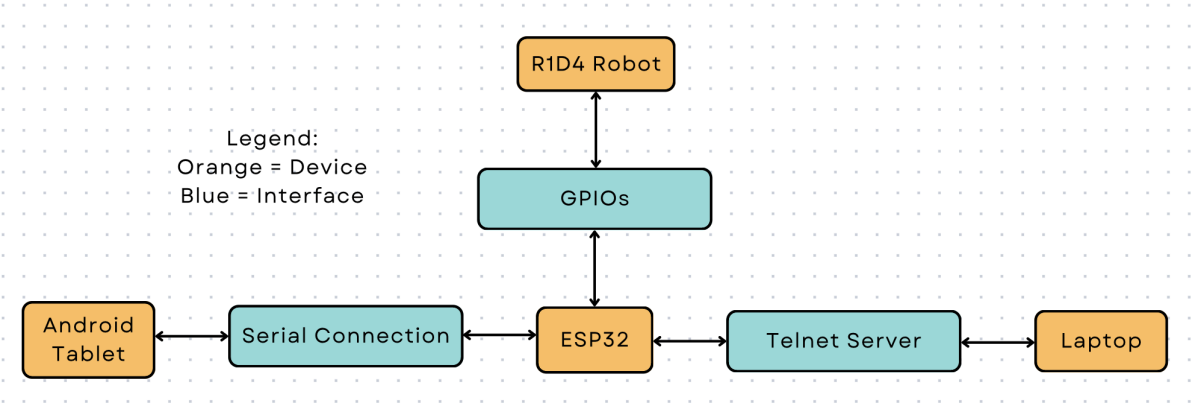


Figure 2: Architectural Overview of System Components

3.2.1 Async Storage

The @react-native-async-storage/async-storage is a package used for simple key-value data storage on a device [4]. It functions similarly to local storage and allows apps to persist small amounts of data across sessions. In our codebase, it's used to store and retrieve scheduled delivery data, allowing the app to maintain delivery states (pending, sent and received) even after the app is closed or restarted.
The react-native-uuid package is a package used to generate Universally Unique Identifiers, a unique ID for each delivery object that is saved to Async Storage. This id is a string usually 36 characters long (128 bits) and ensures that every delivery record has a globally unique identifier, which prevents collisions or overwrites when managing deliveries [5].

4. Requirements and/or Problems

4.1 Requirements

The requirements of our project was to design a comprehensive navigation system for the HoverBot, keeping in mind the existence of other robots. We wanted the user to select a point on a live camera feed and then have the robot autonomously navigate towards it. We needed the robot to be recognizable through common computer vision techniques. Additionally, the system required some way to measure the robots position relative to a grid drawn out by the user. Most importantly, the computer navigation system had to be created in a way so that future groups could easily add other robots to the system.

4.2 Constraints

There were several constraints to consider in this project. Firstly, we were constrained by the existing hardware on the HoverBot system. Although for the most part this did not cause an issue as the hardware system worked decently well, this was something we certainly had to keep in mind. Because of this constraint, when the hardware failed halfway through the project, we did not have enough time to

redesign the system so we were forced into finding suitable replacement parts that worked within the existing structure.

Similarly, we were constrained by the existing software setup. The HoverBot was given to us with an existing software architecture. This included the REST API, TCP server and the C++ code running directly on the ESP32. When we built our computer vision system, we had to make sure our system could issue the correct HTTP requests to the REST API. Also, because our system required extensive computer vision work, we were compelled to develop our navigation system using Python. This is more of a soft constraint as there are certainly other languages that support the computer vision libraries we use. Nevertheless, Python certainly stood out as the best candidate as it supported issuing HTTP requests and the OpenCV computer vision platform.

Another constraint we encountered was the environment/room the robots needed to operate in. Since the robots were based in a lab in Trottier, we were constrained by the lighting conditions in that room. This constrained us on the techniques we could use to find the HoverBot within the camera frame. For example, we could not use a simple black and white thresholding technique to separate the black floor from the rest of the image. This was because when the camera feed was turned to black and gray, the robot/floor were too washed out by the light to separate them effectively using simple thresholding.

The final constraint we dealt with was the camera. Because our budget was limited we could not afford to purchase a camera that could be solely used for this project. Instead, our system was constrained by the fact that we had to use our personal mobile phones as the system's camera. This constrained the recording/streaming software that we could use.

5. Design and Results

5.1 Design and Implementation of the HoverBot Navigation System

This section details the design and implementation of the visual navigation system for the HoverBot, focusing on the specific algorithms, software components, and design decisions made to achieve strong localization and enable movement towards a user-defined target.

5.1.1 Overarching Design Philosophy and Goals

The design of the HoverBot's navigation system was guided by several key principles, particularly considering its potential role within a larger multi-robot system (the "Swarm"):

1. Device Agnosticism: A primary goal was to create a navigation solution independent of the specific sensors available on any individual robot. Swarm members might have varying capabilities (e.g., different IMUs, sonars, or none). Relying on an external, consistent sensor source ensures that the core navigation logic can be applied uniformly across different robot types.
2. Centralized Perception: To achieve device agnosticism, we adopted a centralized perception model using a single, overhead ("bird's-eye view") camera observing the entire operational area. This external camera acts as the primary sensor for all robots within its view.
3. Scalability and Modularity: The external camera approach inherently supports scalability. By processing the video feed centrally, the system can, in principle, track multiple robots simultaneously by identifying unique markers on each. This avoids complex synchronization issues that might arise if each robot reported its own sensor data independently. The vision processing pipeline was designed with this modularity in mind, even though the current implementation focuses on a single HoverBot.
4. Offloading Computation: Processing sensor data, especially video, can be computationally intensive. Performing the complex vision processing tasks on a dedicated computer connected to

the overhead camera significantly reduces the computational load on the HoverBot's potentially resource-constrained onboard processor, freeing it up for control tasks.

5. User-Friendly Interaction: The system needed intuitive ways for a user to calibrate the system (color and perspective) and define navigation goals (clicking a destination).

5.1.2 System Architecture and Data Flow

The navigation system operates through a pipeline that processes frames from the overhead camera, extracts the robot's state, calculates necessary movements, and communicates commands back to the robot. The major steps are:

1. Camera Initialization and Setup: Establish connection to the overhead camera.
2. Perspective Calibration: (One-time setup) Capture user input to define a known rectangle on the floor and compute a homography matrix (H) for perspective correction. Determine the output frame dimensions based on real-world aspect ratios.
3. Color Calibration: (One-time setup) Capture user input (ROIs) to define the HSV color ranges for the unique markers ("Pink", "Green") on the HoverBot.
4. Main Processing Loop:
    a. Capture Frame from Camera.
    b. Apply Perspective Correction: Warp the captured frame using the pre-computed matrix H to get a top-down view (cv2.warpPerspective).
    c. Detect Markers: Convert the corrected frame to HSV (cv2.cvtColor). For each marker color, apply color thresholding (cv2.inRange) and find contours (cv2.findContours). Identify the correct marker contour (using largest area heuristic) and calculate its center (cv2.boundingRect).
    d. Estimate Robot Pose: If both markers are found, calculate the robot's midpoint (current_robot_position) and orientation (angle_r) using vector math based on the marker centers (draw_robot_box).
    e. Handle Destination and Navigation Logic:
        i. Check for a user-clicked destination (click_event).
        ii. If a destination exists and the navigation thread (go_to_destination) is not running, start it.
        iii. The navigation thread uses the latest current_robot_position, angle_r, and the destination to calculate the required turn (delta) and decide the next command (turn left/right, move forward).
    f. Visualize: Draw detected markers, robot bounding box, orientation arrow, destination point, path vector, and relevant text information onto the processed frame (cv2.rectangle, cv2.polylines, cv2.arrowedLine, cv2.putText, cv2.circle).
    g. Display Frame: Show the annotated, perspective-corrected frame (cv2.imshow).
5. Command Transmission: The navigation thread sends commands ("forward", "turnleft", "turnright") with appropriate parameters (distance/angle) to the HoverBot's designated robot_id via an HTTP POST request (send_command_to_bot using requests).

5.1.3 Detailed Implementation Choices and Justifications

1. Perspective Correction (get_parallax_correction, cv2.warpPerspective):

    - Implementation: A four-point homography approach was chosen. The user clicks the four corners of a known rectangle on the floor in the distorted camera view. cv2.getPerspectiveTransform calculates the transformation matrix H. Instead of warping to a fixed arbitrary size, the output dimensions (output_size) are calculated based on the

real-world aspect ratio of the user-measured rectangle (real_width, real_height) and a fixed output height (1080 pixels). This ensures that the resulting top-down view has a consistent scale related to real-world units (pixels per cm, for example, though not explicitly calculated in the code, the aspect ratio is preserved). cv2.warpPerspective applies H to each frame.

- Justification: Accurate navigation requires calculations in a consistent 2D plane. Perspective correction removes distortion, making distances and angles calculated from pixel coordinates meaningful. Basing the output size on the real-world aspect ratio improves the consistency and interpretability of the resulting view compared to warping to an arbitrary square.

2. Color Calibration and Detection (calibrate_color, detect_objects, cv2.inRange):

    - Implementation: HSV color space (cv2.cvtColor(..., cv2.COLOR_BGR2HSV)) is used for its robustness to lighting variations compared to BGR. A user-driven calibration (cv2.selectROI) allows adaptation to specific lighting conditions and marker colors. The calibrate_color function calculates lower and upper HSV bounds from the selected ROI, adding padding ([15, 50, 50]) to account for minor variations and ensuring values stay within [0, 255] (np.clip). cv2.inRange creates binary masks, and cv2.findContours with cv2.RETR_EXTERNAL finds the outer boundaries of the detected color blobs. The largest contour by area (max(contours, key=cv2.contourArea)) is assumed to be the marker.
    - Justification: HSV improves detection reliability. User calibration makes the system flexible. The largest contour heuristic is a simple and often effective way to filter out noise or small spurious detections. Using two distinct colors ("Pink", "Green") allows for unique identification and orientation calculation. (For multiple robots, unique colors or color combinations would be assigned).

3. Robot Pose Estimation (draw_robot_box):

    - Implementation: The robot's position (current_robot_position) is defined as the midpoint (m) between the centers of the detected pink and green markers. The orientation involves calculating a lateral unit vector (lat) from pink to green, then deriving a perpendicular forward unit vector (fwd = np.array([-lat[1], lat[0]])). The angle angle_r is calculated using math.atan2(fwd[0], -fwd[1]) to get the angle relative to the negative Y-axis (pointing up), adjusted to [0, 360). A bounding box is drawn by defining corners relative to m using lat and fwd vectors and adding padding (pad). NumPy is used extensively for these vector calculations.
    - Justification: Using two markers provides both position (midpoint) and orientation (vector between markers). atan2 correctly handles all quadrants for angle calculation. Calculating the bounding box based on the robot's frame (lat, fwd) ensures it rotates correctly with the robot, providing better visualization than a simple axis-aligned box around the markers.

4. Navigation Logic (go_to_destination, compute_angle_d):

    - Implementation: Navigation runs in a separate thread (threading.Thread) to avoid blocking the main vision processing loop. The user sets the destination via mouse click (click_event). The go_to_destination function calculates the desired angle angle_d from the robot's current position to the destination using compute_angle_d (which also uses math.atan2 for angle calculation relative to the Y-axis). The difference delta = angle_d -

angle_r determines the required turn. A simple proportional control logic is used: if abs(delta) is large (e.g., > 45 degrees), turn by a maximum amount; if moderate (e.g., > 10 degrees), turn by abs(delta) (scaled by 9.6, an empirically determined factor relating angle to motor command units); otherwise, move forward. The robot stops when within a threshold distance (calculated based on frame width).

- Justification: Threading prevents the UI from freezing during robot movement/command waits. The control logic is a basic "turn-then-move" strategy, simple to implement and often effective for point-to-point navigation. Using atan2 for both angle_r and angle_d ensures consistent angle referencing. The destination is cleared (destination = None) upon arrival or manual cancellation ('c' key).

5. Communication (send_command_to_bot):

    - Implementation: A dedicated function uses the requests library to send HTTP POST requests to a specified endpoint (http://localhost:8080/{command}). A JSON payload containing the target robot_id and a command parameter (param - distance or angle) is constructed. The function waits for a 200 OK response from the server (assumed to be running on the robot or an intermediary).
    - Justification: HTTP is a standard, flexible communication protocol. Using JSON provides a structured data format. Targeting a specific robot_id is essential for multi-robot control within the Swarm context. This decouples the navigation computer from the robot's specific motor control implementation (as long as the robot exposes the defined HTTP API).

5.2 Results of HoverBot Navigation

We successfully designed and implemented a solid navigation system for the HoverBot. It tracks the position and orientation of the robot in real-time, corrects for camera feed distortions, and allows the user to choose a destination position before sending move commands to the HoverBot via HTTP requests over a TCP server. The result is reliable and accurate. An example of the HoverBot’s navigation in action can be seen in the figure below. A full demonstration can be found in the YouTube link in the appendix.

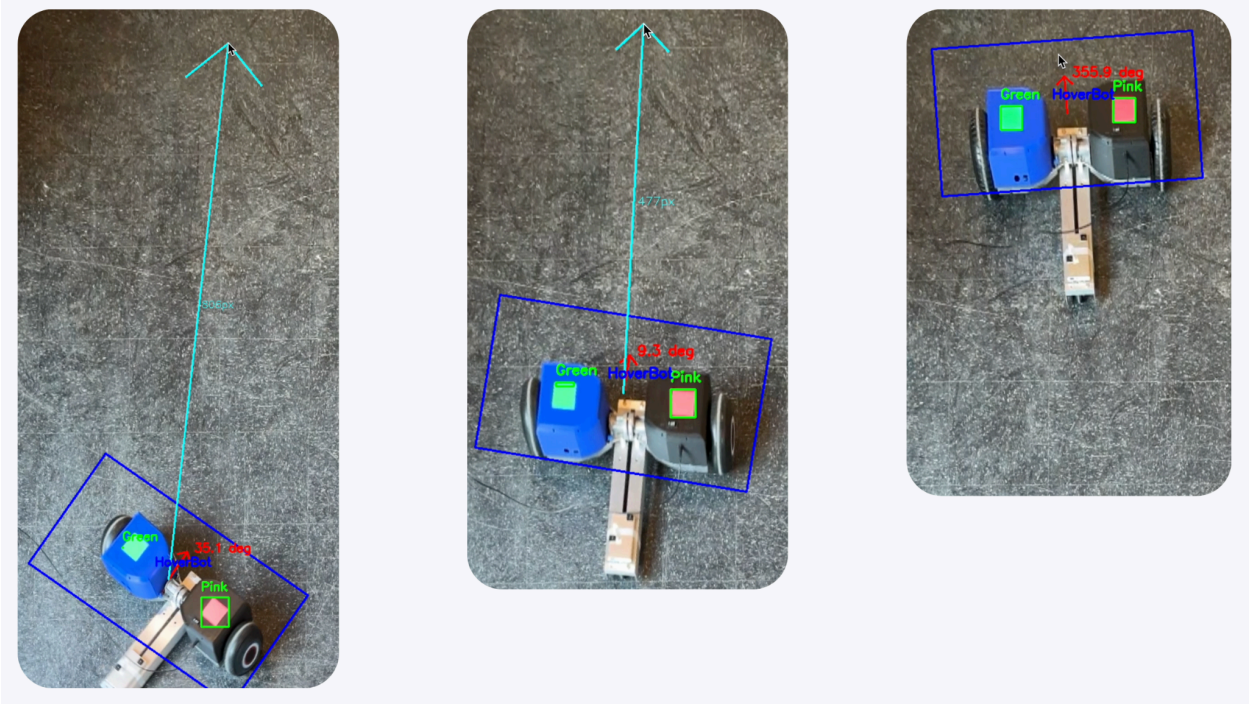


Figure 3: Screenshots from the live camera-feed, showing the HoverBot turning and moving toward the destination.

5.3 Design and Implementation of the R1D4 Interface

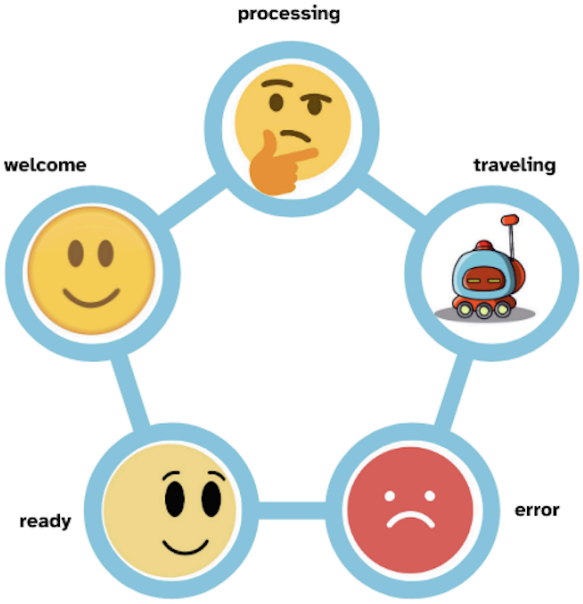


Figure 4: The five states of the robot each having a separate animation, where welcome is the default initial state.

5.3.1 Delivery Status

Each delivery progresses through the following states:

1. Pending: This is the default initial state when a delivery is scheduled by a user. Pending deliveries appear on the *Manage Deliveries* page under the 'Pending' section.
2. Sent: The delivery transitions to this state when it is within five minutes of the scheduled time. At this point, it can no longer be edited or deleted (not displayed under 'Pending' anymore).
3. Confirmed: This state is reached when the recipient confirms the delivery on the *Confirm Pickup* page. Once confirmed, the delivery appears on the *Manage Deliveries* page under the 'Completed' section.

5.3.2 USB Serial Package

The android application establishes a USB serial connection using the react-native-usb-serialport-for-android package to enable communication between the app and a connected USB device, the ESP32 which is connected to the tablet. The connection setup involves listing available USB devices, requesting user permission, and opening the first detected device with specific serial parameters like baud rate and parity [6]. Once the connection is established, the app can send and receive data through the USB port.

Outgoing messages, the scheduled delivery information is formatted, converted to hexadecimal from ASCII, and sent automatically based on a background timer. The react-native-background-timer package is used for periodic background execution by checking/polling scheduled deliveries every minute (to find deliveries scheduled for less than 5 minutes than current time), even when the app is in the background.

Incoming data, the state of the robot, is received in hexadecimal format, decoded to ASCII, and interpreted based on predefined command mappings (processing: 1, error: 2, traveling: 3, ready: 4, welcome: 5), which are used to update the application's global state and display the corresponding animation.

5.3.3 Design Decisions for the R1D4 App

Async Storage over Traditional Databases

Async Storage advantages' include ease of use, asynchronous API (which avoids blocking the main thread) that persists data across app restarts without needing internet connectivity or backend infrastructure, and cross-platform support for both iOS and Android.

Unlike traditional databases that require complex setup, schema management, and external dependencies, AsyncStorage allows for quick key-value data storage using a straightforward API and integrates smoothly with React Native. However, its downsides include lack of encryption by default, limited performance for large-scale or frequent read/write operations, and no built-in support for querying or indexing data like a full database.

Async Storage has a recommended usage limit of 2 to 6 MB, making it best suited for lightweight data rather than large files or media [7]. Nevertheless, it was suitable for our purposes, as each delivery object is stored as JSON formatted string which on average takes about 140 bytes per delivery. Hence, the async storage can store up to 44,000 such deliveries which more than serves our purposes, making it a good choice for storing small to moderate amounts of non-relational data.

Editing and Deleting Pending Deliveries

One of the challenges we encountered early on was managing edits and deletions of delivery information. Initially, delivery data was sent to the robot as soon as a delivery was scheduled regardless of its actual scheduled date and time. However, in this setup, if a user wanted to edit or delete a delivery, we would need to retrieve that specific delivery from the robot, modify it, and resend it. This approach would

require the robot to store all delivery data locally, consuming additional memory, processing power, and other resources.

To address this issue, we revised our delivery-sending logic. Instead of immediately sending delivery information upon scheduling, the system now polls for pending deliveries every minute. It checks whether any deliveries are scheduled to occur within the next five minutes and only those are sent to the robot. With this change, edits and deletions can only be made more than five minutes before the scheduled delivery time.

Any updates made within the app are reflected in the local data storage, and only the final, necessary delivery information is transmitted to the robot. This includes the ID, date, time, floor, and room, just the data required for execution. The complete delivery details (such as sender, recipient, item, quantity, and notes) are retained within the app alone. This approach minimizes computational load and memory usage on the robot, ensuring the overall system and communication remain efficient and lightweight.

5.4 Results of R1D4 App



Fig 4: The four main pages of the app, the Home page (top-left), Send Delivery page (top-right) Manage Deliveries page (bottom-left) and Confirm Pickup page (bottom-right).

5.5 Setup Instructions for HoverBot Navigation System

5.5.1 Prerequisites

Ensure you have the following installed:

- Python 3.x
- OpenCV (opencv-python)
- NumPy (numpy)

- Requests (requests)

For Camera Feed:
- Camo Studio (Mac/Windows App) - If using an iPhone/Android phone as the camera via Camo.
- Camo Camera (iPhone/Android App) - Companion app for the phone.

WiFi / personal hotspot:
- 2.4 GHz, non-enterprise

5.5.2 Hardware Initialization

For setup/checking the hardware of the HoverBot, please refer to this document, specifically the sections on the motor control units, ESP32 control board and power system. It goes over all the necessary information for debugging hardware and was extremely helpful for the setup.[8]

5.5.3 TCP Server

Running the TCP Server

1. Connect your laptop to the same network which you entered for the ssid and password
2. Open the TcpServer directory and cd into it.
3. Might have to do the following: pip install -r requirements.txt and chmod +x gradlew
4. Build the project using: ./gradlew build or ./gradlew.bat on Windows
5. Then run this command: ./gradlew bootrun
6. Now the server should be connected to the HoverBot. The terminal should display something like "added robot with id 1" to indicate this (If using a personal hotspot make sure the phone is kept near the robot).
7. To test if it is working, follow the onscreen commands to list the robots and send a small move command.

Troubleshooting: If the robot has been connected for quite some time or if it disconnected from power, it might automatically add a new ID so make sure to use that instead henceforth.
For iPhone users using hotspots, you might have to open your phone to the hotspot page and make sure you have Maximum Compatibility on and bring it near the robot. If it still doesn't connect, unplug the robot and plug it again to power after doing this.

5.5.4 Python Script

Installation: PyCharm (or another IDE with virtual environment support) is recommended. Clone the repository and the IDE may prompt you to install required packages. Otherwise, install manually using: pip install opencv-python numpy requests.

Setup:
1. Configure Script: Open the Python script (e.g., main.py or similar) and set the correct robot_id global variable near the top (e.g., robot_id = 5). Adjust the cv2.VideoCapture(2) index if camera '2' is not the correct one (try 0, 1, etc.).
2. Open Camo Camera on your phone and Camo Studio on your computer. Ensure they are connected (e.g., via WiFi or USB) and the phone's camera feed appears in Camo Studio. Select the correct device in Camo Studio if needed.
3. Prepare Calibration Area: Have an area of the floor with known dimensions within the camera's view. You will need its width and height.

Running the System:

1. Start the robot's control server application.
2. Start the camera feed (e.g., launch Camo Studio/Camera if used).
3. Run the main Python script from your terminal or IDE: python navigation.py
4. Enter Real Dimensions: The program will prompt you in the console to enter the real-life width and height of the floor (e.g., in cm). Enter these values.
5. Perspective Calibration: A window titled "Parallax Calibration" will appear. Follow the console prompt: Click precisely on the four corners of the rectangle in order: top-left, top-right, bottom-right, bottom-left. Circles will mark your clicks. Once four points are clicked, this window will close.
6. Color Calibration: The "Calibration" window will show the live (still distorted) feed. Follow the console prompts: When prompted for "Select pink square", press c. A crosshair will appear. Click and drag to select a region entirely within the pink marker on the HoverBot. Press Enter to confirm. Repeat the process when prompted for "Select green square" (press c, select the green marker, press Enter).
7. Main Operation: A window titled "Object Tracker" will appear, showing the perspective-corrected video feed. The system will automatically detect the HoverBot, draw its bounding box and orientation arrow (angle_r). Click anywhere in this window to set a navigation destination. A red target circle and a cyan line from the robot to the target (with distance) will appear. The go_to_destination logic will start in the background, sending commands to the robot via HTTP to make it turn and move towards the destination. Observe the robot's movement. Console output will show server responses and navigation status ("Reached Destination!").
8. Press q in the "Object Tracker" window to quit the program.

Controls:

- Mouse Left Click (Perspective Calibration Window): Select the 4 corner points of the calibration rectangle.
- Mouse Left Click (Object Tracker Window): Set the navigation destination point.
- c (during Color Calibration): Initiate ROI selection mode (press before dragging).
- Enter (during Color Calibration): Confirm the selected ROI.
- c (during Main Operation): Manually cancel the current navigation destination and stop the robot's movement towards it.
- q: Quit the program.

Configuration & Tuning - You may need to adjust these parameters within the script:

- robot_id: Set this to match the ID expected by your robot's control server.
- cv2.VideoCapture(index): Change index (e.g., 0, 1) if the camera feed doesn't open.
- pad = 100 (in draw_robot_box): Adjusts the padding around the detected markers for the visual bounding box.
- threshold = frame_width / 10.0 (in go_to_destination): Controls how close (in pixels) the robot needs to be to the destination to be considered "arrived".
- Angle/Forward parameters in go_to_destination (e.g., 45 * 9.6, 300): These values determine the magnitude of turns and forward movements sent to the robot; they may need tuning based on the robot's physical characteristics and the command interpretation by its control server. The 9.6 factor is based on the newest version of the HoverBot and converts degrees to motor command units.
- HSV padding [15, 50, 50] (in calibrate_color): Adjust if color detection is too strict or too loose.
- HTTP Endpoint http://localhost:8080/ (in send_command_to_bot): Change if your robot's server runs on a different address or port.

Notes/Troubleshooting:

- Lighting: Consistent lighting is crucial for reliable color detection. Significant changes in light may require recalibration. Ensure the markers are well and evenly lit.
- HTTP Server: Ensure the robot's control server is running correctly and accessible at the specified endpoint before running the navigation script. Check firewall settings if connection fails.
- Camera Feed: If using Camo, ensure the phone stays connected and awake. Use landscape orientation for the phone camera.
- Calibration Accuracy: Precise clicking during perspective and color calibration is important for accurate tracking and navigation.
- Coordinate System: The system calculates angles relative to the top of the screen (negative Y-axis), with positive angles typically indicating clockwise rotation from that axis. Distances are in pixels within the perspective-corrected view.

5.6 Setup Instructions for R1D4 App

To use the app that is already installed on the tablet follow the steps given below:

1. Connect the ESP32 to your laptop.
2. Open the *R1D4_Telnet.ino* file in the Arduino IDE (download it from arduino.cc).
3. Update the ssid and password variables at the top of the code to match the desired network's name and password.
4. After updating the credentials, upload the code to the ESP32 using the Arduino IDE. Once uploaded, open the Serial Monitor. You should see a message like: “Connected to Wi-Fi.” followed by: “IP Address: xxx.xxx.xxx.xxx”
   Note: Steps 1–4 only need to be done once for each new network connection.
5. Ensure that your laptop or PC is connected to the same network used in the previous steps. If you're using a mobile hotspot, enable Maximum Compatibility mode.
6. Now, physically connect the ESP32 to the tablet via USB. (The app does **not** require Wi-Fi or Bluetooth)
7. In the terminal or command prompt of your laptop, enter the following command (replace xxx with the IP address you obtained earlier, and 23 is the port number):
   telnet xxx.xxx.xxx.xxx 23
8. You should see “Connected to xxx.xxx.xxx.xxx” and do **not** close the terminal or terminate the process while the app is running.
9. Open the R1D4_interface_app on the tablet. When prompted to allow USB access to the UART device, tap **OK**. The app may take around 20 seconds to start.
10. In the terminal, type any of the following commands to update the robot's status and trigger a new animation on the app: processing, traveling, error, ready, welcome (the startup expression). Each command should display the corresponding animation on the tablet.
11. If the animation does not appear, simply close and reopen the app.
12. Finally, the app is connected to the ESP32 via USB and the ESP32 is connected to the laptop (client) and is ready to use.

Note: When you make changes to the codebase and want to debug the app using Android Studio and the tablet, make sure the tablet is connected to the same network. Use ‘Wireless Debugging’ and run the command npx expo run:android. Do not use the Expo Go app to debug, as it doesn’t support some of the required packages and will likely result in errors.

6. Impact on Society and the Environment

6.1 Use of Non-Renewable Resources

6.1.1 HoverBot Navigation System

Our work focused on developing a vision-based system using existing hardware, including a prebuilt HoverBot, a laptop, and a Continuity Camera. By relying on software-based tracking and avoiding the need for additional infrastructure, we minimized further use of non-renewable materials. While no new hardware was introduced (besides the replaced components), the system still relies on electronic components with limited lifespans that may eventually require careful disposal or recycling. In the future, the system could be improved by using more energy-efficient hardware and planning ahead for how components will be reused or recycled once they are no longer needed.

6.1.2 R1D4 Robot User Interface

Our project focused on enhancing the existing touchscreen interface of the R1D4 robot rather than developing the robot itself. Nonetheless, some of the components involved are the Android tablet, sensors and the ESP32 microcontroller depends on non-renewable materials. These components contribute to electronic waste and increase the system's carbon footprint during manufacturing and use. To minimize these effects, we designed the interface with modularity and durability in mind. Our goal was to support long-term use and avoid unnecessary hardware upgrades. For example, we ensured that the app's computational and memory requirements remained lightweight (110 MB) to allow continued use of the existing tablet without performance issues, extending its functional lifespan and reducing waste.

6.2 Environmental Benefits

6.2.1 HoverBot Navigation System

The vision-based navigation system we developed offers environmental benefits by reducing the need for physical infrastructure, such as tracks, guide rails, or external markers, which would require additional materials and energy to produce. By using software-based tracking with OpenCV and a Continuity Camera, and relying on existing devices, the system reduces hardware demands and energy consumption. Compared to traditional robotic systems that rely on pre-installed navigation aids or high-power sensors, our solution reduces both material usage and energy consumption, making it a more environmentally conscious alternative. Though our system was developed on a small scale, this approach could inspire real-world applications such as warehouse automation or delivery systems, where lightweight and adaptable navigation can lower both costs and environmental impact.

6.2.2 R1D4 Robot User Interface

Our project improves hospital operations by automating tasks like medication delivery, reducing the need for staff to perform repetitive transport duties. Although the robot performs one delivery per trip, it still contributes to more efficient workflow and task management. This may lead to small but measurable reduction in the overall energy footprint. Additionally, the interface's modular design supports software updates without the need for frequent hardware replacements, which helps to extend the system's usable life. This helps reduce the amount of discarded electronic equipment over time and supports a more sustainable technology deployment model.

6.3 Safety and Risk

6.3.1 HoverBot Navigation System

During development, our team faced minor safety risks while working with the HoverBot's electrical components, such as verifying wiring, handling power converters, and testing motor behavior.

We addressed these risks by carefully following hardware testing procedures, checking power connections, and ensuring safe handling of all components during debugging. While our system currently does not include advanced safety features, our testing process prioritized preventing unintended behavior or hardware damage. For future teams, we would suggest adding features like obstacle detection or emergency stop functions to help enhance the safety and reliability of the system.

6.3.2 R1D4 Robot User Interface

During development, our primary focus was on the design and implementation of the Android application, as well as establishing reliable communication between the tablet, the ESP 32 microcontroller, and the Telnet server. Although we did not directly handle the robot's hardware, there were still safety considerations related to how commands were sent from the app. Mistakes in formatting or logic could lead to unintended behavior if the robot misinterpreted a command. To address this, we focused on building a reliable and intuitive interface that clearly displays the robot's state through animated expressions and status indicators. This helps users understand the robot's current state and delivery progress through animated expressions and straightforward navigation, reducing the chance of miscommunication or user error. This is especially important in healthcare environments where reliability is essential.

6.4 Benefits to Society

6.4.1 HoverBot Navigation System

Our HoverBot Navigation System shows how low-cost tools like computer vision and consumer hardware can support flexible, autonomous navigation. This approach has the potential to benefit society by improving automation in areas such as warehouse logistics or remote delivery, where reducing manual labour can enhance safety, increase efficiency, and ultimately improve quality of life for workers. The project also lays the groundwork for future development, including extending the vision system to additional robots like the TankBot and BoeBots. These enhancements point toward broader deployment opportunities and may bring us closer to scalable, practical robotics solutions that will offer real-world impact.

6.4.2 R1D4 Robot User Interface

The R1D4 robot and its interface can offer notable benefits in healthcare settings. By automating routine delivery tasks, it reduces the physical workload on medical staff and allows them to focus more on patient care. This can lead to improvements in service quality and staff well-being. From an economic standpoint, hospitals may experience lower operational costs and greater efficiency. The robot also provides a practical solution for healthcare workers with physical limitations who may be unable to perform delivery tasks. In this way, the system supports both accessibility and equity in the workplace.

7. Report on Teamwork

7.1 Team Collaboration

For the first part of our Capstone Project, our team met regularly to discuss progress and make key decisions, such as choosing an Android application over a website and selecting a tablet. We split into two subgroups: Paolo and Joey handled backend and hardware, while Anjali and Noorjahan worked on the user interface. This division allowed for steady, parallel development across all areas.

For the second part of the project, we continued with regular meetings and structured workflow to manage tasks like hardware setup, computer vision, and navigation logic. Each team member stayed

involved across different components, and we relied on clear communication and hands-on testing to troubleshoot issues and integrate the system effectively

7.2 Individual Contributions

Paolo Bizzarri: In regards to the R1D4 project, Paolo's main contributions revolved around establishing communication between the hardware and software components. He worked on getting the old code from the previous group running on the ESP32, setting up serial communication to relay messages between the ESP32 and a laptop, and enabling serial communication between the tablet and ESP32. He also created a test application to demonstrate communication between the Telnet server, tablet, and ESP32 and confirmed that the application worked on the tablet hardware. Additionally, Paolo integrated the USB/Telnet communication system in the final product. This involved editing the existing structure from the test application. This allowed the user to change the animations on the homepage along with facilitating the schedule information being received on the telnet server. With the HoverBot, Paolo worked on dealing with hardware related bugs and getting the robot set up and operating in a state similar to where it was when the last group left it. This involved replacing both the motor control units on the board and the 36v to 12v DC converter. He also re-wired the power system so that the hover bot could source current directly from a charger. This was due to the fact that the battery failed on the robot late into the project. He also worked on setting up the TCP server with Anjali.

Since it was helpful for our team to speak with the previous group for setup issues, I will leave my phone number here if any future groups need to contact me. +1 416 845 6620.

Joey Hershkop: For the R1D4 project, Joey initially contributed to the systems integration logic and how the R1D4 robot, ESP32, and interface would end up connecting. He then contributed to the UI state flow and how the various pages and menus of the app would connect. He helped design and lay out the frontend visually in Figma. Later, he got the previous R1D4 code preliminarily up and running on a test ESP32, and then initialized the telnet server on a local network. For the HoverBot navigation system, Joey conceived of and decided on the camera-based navigation system. He designed and implemented the preliminary OpenCV system that was capable of recognizing and tracking the HoverBot's position and orientation based on coloured markers. He implemented parallax and aspect ratio correction logic, along with the UI to interact with the system. He worked alongside Anjali to implement the logic in the go_to_destination function, and excessively tested the navigation system in a variety of environments to ensure accurate and robust operation.

Anjali Singhal: Anjali contributed to both the design and development of the project. She began by creating the initial Figma designs and determining the layout of the app pages. She was responsible for selecting the frontend framework, setting up the project structure, and initializing the Git repository. This process involved researching and experimenting with various frontend technologies before ultimately choosing React Native with Expo and JavaScript. She worked on the frontend and backend implementation of all pages, which included core functionalities such as sending, deleting, confirming, and editing deliveries. She also added the Async Storage mechanism to ensure persistent data storage across sessions. For the HoverBot navigation system, Anjali developed understanding of the TCP server and contributed by identifying and fixing bugs in the server code. She integrated the REST API requests into the navigation flow to dispatch appropriate commands to the robot. Additionally, she was involved in designing and implementing some of the logic for the navigation algorithm and testing its execution on the HoverBot.

Noorjahan Abonty: Noorjahan's contributions spanned both interface design and hardware testing throughout the two phases of the project. For the R1D4 Robot User Interface, she assisted in the

selection and setup of the software framework by exploring configurations for React Native and Expo, and by conducting a detailed comparison between building a website versus an Android app. Her analysis played a key role in guiding the team's final decision. She contributed to the development of the "Manage Tasks" and "Confirm Pickup" pages of the Android application by completing their preliminary setup, implementing access to the global delivery array, and creating the confirm button functionality. Additionally, she redesigned the task layout and added support for editing, deleting, and confirming deliveries to improve task management within the app. For the HoverBot Navigation System, Noorjahan was involved in the initial setup and wiring verification of the HoverBot hardware. She manually tested each pin by disconnecting and observing expected motor behavior, and confirmed proper grounding and power connections from the onboard converters. She followed hardware testing procedures to verify correct wheel responses and identify issues such as wiring faults or misconfigured brake and drive pins, ensuring that the ESP32 and motor controllers were functioning as intended.

8. Conclusion

We made significant progress by the end of the project, notably developing a fully functional Android application from scratch using React Native, Expo, and JavaScript, technologies that were initially unfamiliar to most of our team. This experience pushed us out of our comfort zones and strengthened our ability to learn new frameworks.

On the hardware side, we successfully picked up where the previous group left off with the HoverBot. Through careful analysis of their work and understanding of the system architecture, we were able to build upon their foundation and implement a working object tracking and navigation system. Our project spanned a wide range of technical concepts, particularly in terms of communication across multiple systems and interfaces. We worked extensively with Telnet, ESP32s, TCP servers, and REST APIs, which demanded a continuous process of exploration, problem-solving, and integration. These layers of inter-device communication taught us the importance of protocol compatibility, real-time responsiveness, and modular system design.

Overall, the project combined a mix of skills which included mobile development, network communication, and robotics. It challenged us to stay adaptable, think critically, and work as a team to deliver a complex system from end to end.

8.1 Next Steps for HoverBot Navigation

8.1.1 Hardware

Regarding the next steps for the hardware on the HoverBot, the main focus is the power system. As the system stands right now, the robot is receiving power directly from a 2amp charger that is being fed into a 36v to 12v convertor. It is bypassing the switch as well. This was a temporary measure as the power system broke down approximately ten days before our demo day. For the next group, they will need to rewire the power system to include the switch again and also purchase two small fuses that will protect the battery from breaking down. Additionally, they will need to replace the battery on the robot.

Here is an image of the power system. The red lines indicate the temporary changes of mind while the black lines indicate how the power system is designed to work.

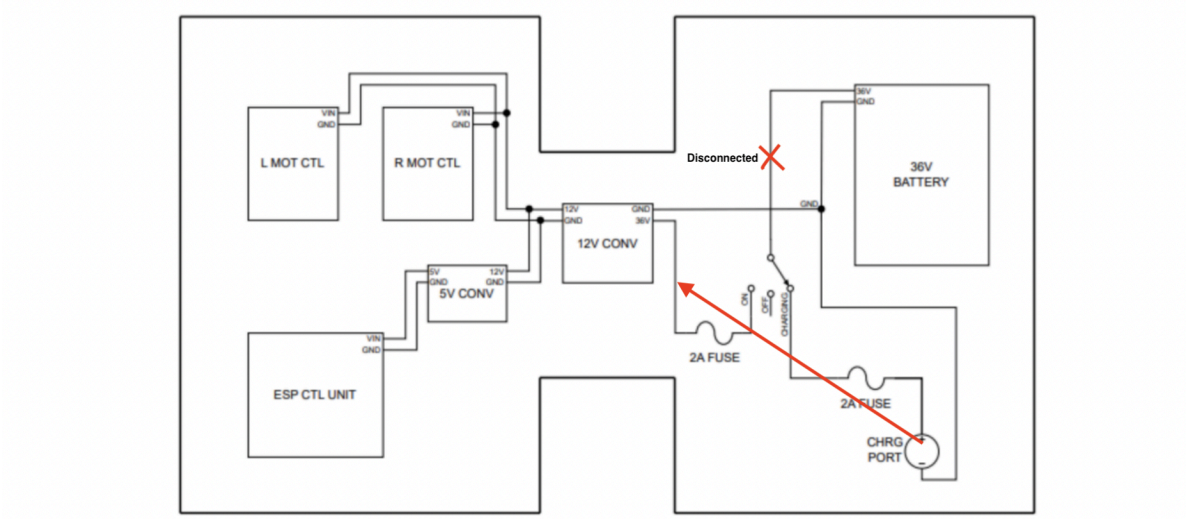


Fig 5: Hoverbot Power system with Current Edits.

After implementing these changes, the robot should work as intended. If you suffer from any other issues regarding the hardware setup, please reference the following document created by the previous capstone group.

8.1.2 Software

The most impactful next steps for the navigation software involve expanding this system to manage multiple robots concurrently, fulfilling the broader vision of the Swarm project, and addressing the inherent challenges that arise in multi-robot scenarios, primarily collision avoidance.

I. Multi-Robot Tracking and Control: The current system is designed with scalability in mind due to its reliance on an external camera, but the implementation currently tracks only one predefined set of markers (Pink/Green) associated with a single robot_id. Expanding this involves:

1. Unique Robot Identification: Assign unique identifiers to each robot in the Swarm. This could be achieved by:
    - Unique Color Combinations: Assigning distinct pairs of colors (e.g., Blue/Yellow, Red/Cyan) to each robot. This requires extending the color calibration process to handle multiple color sets.
    - Pattern-Based Markers: Utilizing more robust fiducial markers like AprilTags or ArUco markers (supported by OpenCV's aruco module). Each tag has a unique ID that can be directly read by the vision system, offering greater robustness than color alone, especially under varying lighting or with many robots. This would replace the current color detection module.
2. Software Architecture Modifications:
    - Dynamic Robot Management: Implement mechanisms to register active robots and their corresponding markers (colors or tag IDs).

- Generalized Detection: Modify the detect_objects function (or replace it if using ArUco) to iterate through all known marker sets/tags within a single frame, identifying all visible robots.
- Multi-Pose Storage: Store the calculated pose (position and orientation) for each detected robot, likely using a dictionary keyed by robot_id.
- Parallel Navigation Logic: Adapt the go_to_destination logic. This might involve launching separate navigation threads for each robot tasked with reaching the destination or implementing a central coordinator that manages commands for all robots based on their individual states and the common goal.
- Targeted Command Dispatch: Ensure the send_command_to_bot function correctly routes commands to the appropriate robot_id based on which robot's navigation logic is generating the command.
- Enhanced Visualization: Update the display to simultaneously show bounding boxes, orientation arrows, IDs, and potentially individual paths for all tracked robots.

II. Collision Avoidance: Operating multiple robots within the same physical space necessitates a robust collision avoidance strategy to ensure safety and efficient operation. The centralized vision system is advantageous here, as it has simultaneous knowledge of all robots' positions. Potential approaches include:

1. Reactive Collision Avoidance:
    - Concept: Implement simple rules based on proximity. If two robots come within a predefined safety radius of each other, one or both robots execute a predefined maneuver (e.g., stop, slow down, yield priority based on ID).
    - Implementation: Add a processing step after pose estimation that iterates through all pairs of detected robots. If the distance between any pair is below the safety threshold, override their navigation commands with the avoidance maneuver.
    - Pros/Cons: Relatively simple to implement but can lead to suboptimal paths, oscillations, or deadlocks (robots blocking each other).
2. Proactive / Predictive Collision Avoidance:
    - Concept: Use current position and velocity (or intended movement direction) to predict potential collisions in the near future. Techniques like Velocity Obstacles (VO) or Artificial Potential Fields (APF) could be adapted.
    - VO: Calculates the set of relative velocities that would lead to a collision within a time horizon and commands the robot to choose a velocity outside this set.
    - APF: Models robots as repulsive fields and the goal as an attractive field. The robot moves based on the gradient of the combined field.
    - Implementation: Requires calculating or estimating robot velocities/intentions. The avoidance logic becomes more complex, adjusting commanded velocities or turning angles proactively to steer robots away from predicted collision courses.
    - Pros/Cons: Can lead to smoother, more efficient multi-robot motion but requires more complex calculations and careful parameter tuning.
3. Integration: The collision avoidance module would need access to the multi-robot pose data. It would act as an intermediary layer, potentially modifying the commands generated by the basic go_to_destination logic before they are sent to the robots via send_command_to_bot.

8.2 Next Steps for R1D4 App

While the current delivery storage mechanism works perfectly well as per the small to moderate data storage requirements, there are some aspects that can be optimized if need be for better performance and to speed up execution. These include:

A. Storing delivery data in a sorted manner by sorting as per the date and time such that the deliveries with the earliest date and time are at the beginning followed by the ones later on.
B. Every minute instead of doing a linear search on all the pending deliveries, perform a binary search on the sorted delivery data to find all deliveries for the given time
C. Switching to a traditional relational database to scale up and store more data or even media like images
D. While performing some last minute testing we discovered a bug on the *Manage Deliveries* page for the edit functionality, essentially the update date and time does not update the deliveries correctly internally and such deliveries are not sent at all. However, editing the other fields works fine.

Finally, incorporate the actual R1D4 robot code to have multiple states such that it follows this desired workflow. The following steps outline the desired workflow, detailing how the app and Telnet server and the ESP32 together simulate the robot's behavior and what is expected from the robot. The robot should be able to transition between five states: 'welcome', 'processing', 'traveling', 'error', and 'ready'.

1. The user schedules a delivery via the *Send Delivery* page and can edit or delete 'Pending' deliveries or delete 'Completed' deliveries through the *Manage Deliveries* page.
2. Once a delivery is scheduled, it is sent to the robot five minutes before the scheduled delivery time. The Telnet connection typically takes around 2 minutes to receive and display the delivery information in the terminal.
3. Upon receiving the delivery data, the robot should enter the 'processing' state and send a confirmation signal back to the app. This behavior is currently simulated by manually entering 'processing' in the terminal, which triggers the corresponding animation in the app.
    a. During the processing phase, the robot should determine the route to the destination room and floor (assuming it has access to a hospital map and navigation capabilities).
4. After successfully computing the path, the robot should switch to the 'traveling' state and send another signal to the app. This is simulated by typing 'traveling' in the terminal, rendering a new animation.
5. Assuming the robot has reached the destination, at this point, the delivery no longer appears under the 'Pending' section in the *Manage Deliveries* page. Instead, it is shown on the *Confirm Pickup* page.
6. Once the recipient confirms pickup on the *Confirm Pickup* page, the delivery is marked as completed and moves to the 'Completed' section of the *Manage Deliverie*s page.
7. The completed delivery acts as a log entry for hospital records but can also be permanently deleted from the system by pressing the delete button
8. After successful delivery and confirmation, the robot should enter the 'ready' state to indicate it's prepared for the next task. This is simulated by entering 'ready' in the terminal, triggering the relevant animation.
9. If the robot encounters an issue at any time and cannot complete the delivery, it should enter the 'error' state. This is similarly simulated by typing 'error' in the terminal to render the error animation.

This workflow ensures efficient communication between the robot and app while enabling clear visualization of the robot's status through animations.

9. References

[1] mad-ee.com, "Easy Inexpensive Hoverboard Motor Driver - MAD-EE," *MAD-EE - I Make Electrons Do My Bidding*, Oct. 25, 2021. https://mad-ee.com/easy-inexpensive-hoverboard-motor-controller/

[2] StreamSets Inc., "TCP Server," StreamSets Documentation, Apr. 2025. [Online]. Available: https://docs.streamsets.com/platform-datacollector/latest/datacollector/UserGuide/Origins/TCPServer.htm

[3] "OpenCV: OpenCV modules." https://docs.opencv.org/4.x/index.html

[4] React Native Community, "*@react-native-async-storage/async-storage*", npm, [Online]. Available: https://www.npmjs.com/package/@react-native-async-storage/async-storage [Accessed: Apr. 11, 2025].

[5] P. Leach, M. Mealling, and R. Salz, *A Universally Unique Identifier (UUID) URN Namespace*, RFC 4122, Internet Engineering Task Force (IETF), July 2005. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4122

[6] M. Mik3y, *usb-serial-for-android*, GitHub repository, https://github.com/mik3y/usb-serial-for-android, accessed Apr. 11, 2025.

[7] React Native Community, "Storage limits," *React Native AsyncStorage Documentation*, Apr. 2025. [Online]. Available: https://react-native-async-storage.github.io/async-storage/docs/limits. [Accessed: Apr. 11, 2025].

[8] Ghosh, Pinak, "*Prometheus Swarm HoverBot Documentation*", Apr 2025 [Online]. Available: https://drive.google.com/file/d/1I3W-P_5KOXJP6YtNQNTpK6jNLg2feTBn/view. [Accessed: Apr 11 2025]

10. Appendices

HoverBot Demo link: https://youtu.be/0KpnwpMZ4bQ